Contract Farming Management System - Farmer and Merchant Agreement Platform

A PROJECT REPORT

Submitted by

Sarikha RV (310624150041)

in partial fulfillment for the award of the Degree

of

BACHELOR OF ENGINEERING

in

DEPARTMENT OF COMPUTER SCIENCE AND DESIGN

2311CSE503R - FULL STACK DEVELOPMENT



EASWARI ENGINEERING COLLEGE,

(AN AUTONOMOUS INSTITUTION) – AFFILIATED TO ANNA UNIVERSITY, CHENNAI

OCTOBER 2026

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this mini project report titled **"Contract Farming Management System – Farmer and Merchant Agreement Platform"** is the Bonafide work of **"Sarikha RV (310624150041)",** who carried out the project work under my supervision.

HEAD OF THE DEPARTMENT

Dr. J. Deepa

SUPERVISOR

Ms. Divyanjali

EASWARI ENGINEERING COLLEGE

Certificate of Viva-voce-Examination

This is to certify that the Viva Voce Examination of the Project Work entitled "**Contract Farming Management System – Farmer and Merchant Agreement Platform**" submitted by Ms. **Sarikha RV**(310624150041) in partial fulfillment of the requirements for the award of the degree of **Bachelor of Engineering in Computer Science and Design** was conducted on 26.10.2026 in the Department of Computer Science and Design, Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner

Name :
Designation :
Address :

External Examiner

Name :
Designation :
Address :

ABSTRACT

The Contract Farming Management System is a web application that lets farmers and merchants agree on a crop price before sowing, record that agreement digitally, and follow it through to payment. Small farmers in India often sell without a written agreement, depend on middlemen for price and payment, and have no proof of what was promised when a dispute arises. Merchants face the opposite problem: they cannot see the field, so they cannot judge whether the crop will arrive in the agreed quantity and grade.

The system is built on the MERN stack: React (Vite) for the interface, Node.js and Express for the REST API, and MongoDB Atlas with Mongoose for storage. Three roles are supported: Farmer, Merchant (Buyer) and Admin. A merchant drafts a contract with a farmer, and the contract becomes active only after both parties sign with their password. The farmer then advances the crop through four stages (Sowing, Growth, Harvest, Delivered) and uploads geotagged field photos taken with the phone camera. The merchant views these photos in a timeline gallery and releases payment in three milestones of 30%, 40% and 30%, strictly in order. The admin verifies new users and resolves disputes. Every sensitive action is written to an audit log.

The interface is designed for low-literacy, mobile-first use. It is available in English, Tamil, Hindi, Telugu and Malayalam, with a spoken guide on the first screen. Access rules are enforced on the server with JWT authentication, role middleware and ownership checks. Payments run through a sandbox provider so that a real gateway can replace it later; no real money moves in this version.

Keywords: contract farming, MERN stack, MongoDB, JWT authentication, digital contracts, milestone payments, multilingual interface, field photo evidence.

ACKNOWLEDGEMENT

We express our sincere gratitude to the Principal, Dr. P. Deiva Sundari, and the Management of our college for providing us with the opportunity to carry out this project work.

We convey our heartfelt thanks to the Head of the Department, Dr. J. Deepa, Department of Computer Science and Design, for the continuous encouragement and support.

We are extremely thankful to our Project Guide, Ms. Divyanjali, Department of Computer Science and Design, for valuable guidance, suggestions, and motivation throughout the completion of this project.

We would also like to extend our gratitude to all the faculty members of the Department of Computer Science and Design for their support.

Finally, we thank our parents and friends for their constant encouragement and help during the course of this project.

TABLE OF CONTENTS

TABLE OF CONTENTS

CHAPTER NO. / TITLE ..... PAGE No.
ABSTRACT i
ACKNOWLEDGEMENT ii
CHAPTER 1 – INTRODUCTION 1
1.1 OVERVIEW 1
1.2 PROBLEM DEFINITION 1
1.3 OBJECTIVE 2
1.4 SCOPE OF THE PROJECT 2
1.5 MOTIVATION 3
CHAPTER 2 – RELATED WORK 5
2.1 LITERATURE REVIEW 5
2.2 EXISTING SYSTEM 6
2.3 PROPOSED SYSTEM 6
CHAPTER 3 – SYSTEM ANALYSIS 8
3.1 SYSTEM REQUIREMENTS 8
3.2 SOFTWARE AND HARDWARE REQUIREMENTS 11
CHAPTER 4 – SYSTEM DESIGN 13
4.1 SYSTEM ARCHITECTURE 13
4.2 MODULE DESCRIPTION 14
4.3 DATA FLOW AND DATABASE DESIGN 15
4.4 CLASS DIAGRAM 16
4.5 SEQUENCE DIAGRAM 17
CHAPTER 5 – IMPLEMENTATION 23
5.1 IMPLEMENTATION OVERVIEW 23
5.2 KEY FUNCTIONS (CODE EXPLANATION) 24
5.3 FRONTEND STRUCTURE AND ROUTING 24
5.4 VALIDATION AND TEST SCENARIOS 24
5.5 DEPLOYMENT AND CONFIGURATION 24
5.6 SUMMARY OF IMPLEMENTATION 24
CHAPTER 6 – CONCLUSION & FUTURE WORK 33
6.1 CONCLUSION 33
6.2 FUTURE ENHANCEMENTS 33
APPENDIX 35
A.1 Project Structure 35
A.2 Backend Package Installation 35
A.3 Database Connection 35

A.4 Environment Variables 35
A.5 User Model 35
A.6 Password Hashing and Registration 36
A.7 Login and JWT Generation 36
A.8 Authentication Middleware 36
A.9 Create API 36
A.10 Read API 37
A.11 Update 37
A.12 Delete 37
A.13 Express Routes 37
A.14 Main Server File 38
A.15 React Login Component 38
A.16 Admin Dashboard Logic 38
A.17 REST API Request and Response Examples 38
A.18 Product Model 38
A.19 Sale Model (Multi-Product Bill) 38
SCREENSHOTS OF THE PROJECT 40
REFERENCES 43

CHAPTER 1
INTRODUCTION

1.1 Overview

Contract farming is an arrangement in which a farmer agrees, before planting, to supply a fixed quantity and grade of a crop to a buyer at a pre-agreed price. The buyer gets a predictable supply of raw material that meets a known specification. The farmer gets a guaranteed market and protection from price falls at harvest. Because the price is fixed before sowing, the farmer can also plan inputs such as seed, fertiliser and labour with more confidence, and can sometimes use the agreement as proof of future income when seeking credit.

In practice, most such agreements in India are verbal or kept on plain paper, which makes them hard to enforce and easy to dispute. A verbal promise made in January is difficult to prove in October, particularly when the market price has moved sharply in the meantime. The party that gains from the movement is tempted to forget the promise, and the party that loses has nothing to show.

The Contract Farming Management System moves the whole agreement online.All records are stored in MongoDB, so both sides see the same history, and sensitive events are copied into an audit log that cannot be edited from the user interface.

The platform is written as a full stack web application. The client is a React single-page application, the server is a REST API written with Node.js and Express, and the data lives in MongoDB Atlas. The design keeps the roles of the three tiers separate so that each can be developed, tested and deployed independently.

1.2 Problem Definition

The problems this project addresses are:

- Verbal or paper agreements give farmers no reliable proof of price, quantity or delivery date.
- Merchants cannot see the field, so they cannot confirm the crop is progressing as promised.
- Payments are delayed or paid in a lump sum at the end, which leaves the farmer carrying all the cost and risk during the season.
- Disputes have no neutral record, so the outcome depends on memory and bargaining power.

- Most farming software assumes English and good literacy, which excludes many farmers.

These problems reinforce one another. Without a record, a dispute cannot be settled fairly; without visible progress, the merchant hesitates to pay early; without early payment, the farmer cannot afford good inputs, which in turn lowers the quality the merchant receives.

1.3 Objective

The main objective is to build a secure, mobile-first web platform that makes contract farming transparent and trackable. The specific objectives are:

- To let farmers and merchants register with a mobile number and sign crop contracts digitally.
- To track crop progress through four stages, allowing only the contract's farmer to advance a stage, one step at a time.
- To let farmers capture and upload field photos with time and location, and let merchants view them.
- To release payment in three ordered milestones (30%, 40%, 30%) linked to contract signing, harvest and delivery.
- To give an administrator tools to verify users and resolve disputes.
- To offer the full interface in English, Tamil, Hindi, Telugu and Malayalam, with spoken instructions.
- To enforce every access rule on the server and keep an audit trail of sensitive actions.

1.4 Scope of the Project

The scope of the project is a web application for managing crop contracts between farmers and merchants.

In scope:

- Registration, login and role-based access for Farmer, Merchant and Admin.
- Contract creation, two-party signing with password confirmation, and status tracking.
- Crop stage updates, field photo upload with camera capture, compression and offline queueing.
- Milestone payments through a sandbox payment service.
- Disputes, notifications, dashboards and an admin console.

- A five-language interface and a pre-login guide.

Out of scope for this version:

- Real money transfer. The payment provider is a sandbox, written so that a gateway such as Razorpay can replace it.
- Legal enforceability of the digital signature. The signature here is a password-confirmed record, not a certified e-signature.
- Automatic crop-health analysis from photos.
- Native mobile applications. The web application is designed to work well on phones.

1.5 Motivation

Agriculture supports a large share of the Indian workforce, yet many small farmers still sell at whatever price the market offers on harvest day. Government policy has recognised the value of written farming agreements, and the central government's Model Contract Farming Act, 2018 was issued as a template for states. What is missing at the field level is a tool simple enough for a farmer with a basic smartphone.

This project is motivated by the idea that a clear contract, visible progress and staged payments can build trust between farmer and buyer, and that the software must speak the farmer's language to be used at all.

1.6 Organisation of the Report

Chapter 2 reviews related policy, platforms and technologies and compares the existing way of working with the proposed system. Chapter 3 lists the functional and non-functional requirements and the business rules. Chapter 4 describes the architecture, modules, database and the main interaction flows. Chapter 5 explains the implementation and the key functions. Chapter 6 reports the testing carried out and the results. Chapter 7 concludes the report and lists future enhancements. The appendix contains the source code and API examples, followed by screenshots and references.

1.7 Organisation of the Report

Chapter 2 reviews related policy, platforms and technologies and compares the existing way of working with the proposed system. Chapter 3 lists the functional and non-functional requirements and the business rules. Chapter 4 describes the architecture, modules, database and the main interaction flows. Chapter 5 explains the implementation and the key functions. Chapter 6 reports the testing carried out and the results. Chapter 7 concludes the report and lists future enhancements. The appendix contains the source code and API examples, followed by screenshots and references.

CHAPTER 2

RELATED WORK

2.1 Literature Review

Contract farming policy. The Government of India issued the Model Agricultural Produce and Livestock Contract Farming and Services (Promotion and Facilitation) Act, 2018 as a template for states. It describes a written agreement between farmer and sponsor, a price fixed in advance, and a dispute resolution authority. The Farmers (Empowerment and Protection) Agreement on Price Assurance and Farm Services Act, 2020 covered similar ground at the central level before it was repealed in 2021. Both documents show what a contract should contain, which this project uses as the basis for its contract fields: crop, quantity, price, grade and delivery date. They also show the importance of a neutral dispute mechanism, which this project models through the administrator role and a recorded resolution note.

Digital agricultural marketplaces. The National Agriculture Market (e-NAM) connects regulated markets online so that traders can bid for produce across states. It focuses on spot trading at the market, not on agreements made before sowing, and it does not follow a crop through the season. The Contract Farming Management System sits at a different point in the value chain: it starts before the seed is sown and ends when the final payment is made.

Farmer information apps. Government and private mobile apps give farmers weather forecasts, mandi prices and crop advice. They inform decisions, but they do not record a commitment between two parties. Their value is complementary: a farmer could consult such an app before accepting the price in a contract. Mandi price display is therefore listed as a future enhancement rather than a core feature.

Web technology for low-resource users. Browser APIs make several features possible without a native app: camera access through getUserMedia, location through the Geolocation API, speech output through the Web Speech API, and offline storage through IndexedDB. Token-based authentication with JSON Web Tokens (RFC 7519) and adaptive password hashing with bcrypt are standard practice for securing REST APIs. The OWASP Top 10 list of web application risks guided the security choices of Chapter 5, particularly broken access control, injection and security misconfiguration.

2.2 Existing System

In the existing way of working, the agreement between a farmer and a merchant is verbal or written on plain paper. Progress is checked by the merchant visiting the field or by phone calls. Payment is negotiated at harvest, often in cash, and an advance is given informally.

Aspect	Existing practice	Consequence
Agreement	Verbal or plain paper	No dependable proof of price, grade or date
Crop monitoring	Visits and phone calls	Costly for the merchant and unverifiable
Payment	Lump sum at harvest, or informal advance	Farmer carries the season's cost and risk
Disputes	Settled by negotiation or local mediation	No record, so the weaker party loses
Language	Mostly English-based software	Many farmers cannot use it

Limitations of the existing system. The existing practice has five weaknesses. First, it is not transparent: the two parties may remember different terms. Second, it is not auditable: no one can later reconstruct who agreed to what and when. Third, it is expensive to supervise: every field visit costs the merchant time and travel. Fourth, it concentrates risk on the farmer, who finances the whole season before seeing any money. Fifth, it excludes farmers who cannot read English, because the few digital tools that exist rarely offer regional languages or spoken help.

2.3 Proposed System

The proposed system replaces these practices with a single web platform. Its main differences are:

- **Digital contracts.** A contract is created by the merchant and becomes active only when both parties sign with their password. The server records who signed and when.
- **Staged progress.** The crop moves through Sowing, Growth, Harvest and Delivered. Only the contract's farmer can advance it, one step at a time.
- **Photo evidence.** The farmer takes field photos in the app. Each photo carries a stage, a note, a timestamp and optional coordinates, and cannot be edited or deleted.

- **Milestone payments.** Payment is split 30% on signing, 40% at harvest and 30% on delivery, and the merchant releases each in order.
- **Dispute handling.** Either party can raise a problem, and the admin resolves it with a recorded note.

2.4 Comparison of Existing and Proposed Systems

Feature	Existing practice	Proposed system
Record of agreement	None or paper	Digital contract with signing time stamps
Progress evidence	Phone call or visit	Geotagged, time-stamped field photos
Payment pattern	Lump sum at the end	30%, 40%, 30% in enforced order
Dispute record	None	Dispute entity with resolution note and resolver
Language support	Mostly English	English, Tamil, Hindi, Telugu, Malayalam

2.5 Feasibility Study

Technical feasibility. Every component of the system uses mature, free and well-documented technology. The MERN stack is widely taught and supported, MongoDB Atlas offers a free tier suitable for a prototype, and the browser APIs used for the camera, location, speech and offline storage are available in current Android and desktop browsers. The development team's skills in JavaScript cover the client and the server, which removes the cost of switching languages.

Economic feasibility. The prototype runs on free tiers: Atlas for the database, Render for the server and Vercel for the client. Development needs only a laptop and a browser. The sandbox payment service avoids transaction fees during development. If the system grows, the recurring costs would be hosting, file storage and payment-gateway fees, all of which scale with usage.

Operational feasibility. The target users are small farmers and small merchants. The interface therefore uses large buttons, icons with labels, regional languages and spoken guidance, and keeps each screen to one task. Admin approval of new users gives the platform a trust layer without requiring formal paperwork at registration.

Legal feasibility. The platform records agreements but does not claim to be a legally certified e-signature service. The report states this limit openly and lists certified signing as a future enhancement, so the prototype does not mislead users about the legal weight of a signature.

CHAPTER 3

SYSTEM ANALYSIS

3.1 System Requirements

Requirements were collected by studying how contract farming works today, by reading the model contract farming law, and by listing what each role needs to do. They are grouped into functional requirements, business rules and non-functional requirements.

Functional requirements

ID	Requirement	Role
FR1	Register with name, 10-digit mobile number, password and village (farmer) or company (merchant)	Farmer, Merchant
FR2	Log in with mobile number and password and receive a JWT	All
FR3	Select and save the preferred language	All
FR4	Verify or approve newly registered users	Admin
FR5	Create a contract with a farmer: crop, quantity, price per quintal, grade and delivery date	Merchant
FR6	Sign a contract after re-entering the password; the contract becomes active when both have signed	Farmer, Merchant
FR7	Advance the crop stage by one step on an active contract	Farmer
FR8	Capture or choose a photo, compress it, attach stage, note and location, and upload it	Farmer
FR9	View the field photo gallery, filter by stage, and open photos full screen	Merchant, Farmer
FR10	Release payment milestones in order: 30%, 40%, 30%	Merchant
FR11	View payment status and invoice numbers	Farmer, Merchant
FR12	Raise a dispute against a contract	Farmer, Merchant
FR13	Resolve a dispute with a note; view all contracts and disputes	Admin
FR14	Receive in-app notifications for signing, stage changes, photos and payments	All
FR15	Remove an uploaded photo	Admin

Explanation of the main requirements. Registration (FR1) collects only what is needed to identify a user in the field: a name, a mobile number that doubles as the login

name, a password, and either a village or a company name. Login (FR2) returns a signed token that the client attaches to every later request. Contract creation (FR5) is restricted to merchants because the merchant is the party offering the price; the farmer then accepts by signing (FR6). Stage advance (FR7) and photo upload (FR8) are restricted to the farmer because only the farmer can truthfully report what is happening in the field. Payment release (FR10) is restricted to the merchant because only the buyer can authorise payment. Disputes (FR12) are open to both parties so that neither is silenced, and are closed only by the admin (FR13) so that neither party can resolve its own complaint.

3.1 Business Rules Enforced on the Server

- A contract is active only after both parties sign.
- A stage can move forward only one step at a time, only by the contract's farmer, and only on an active contract.
- Payment milestones are released only by the merchant, only in order, and only when the stage condition is met: 30% on signing, 40% at Harvest, 30% at Delivered.
- Unverified users can log in but cannot create or sign contracts until the admin approves them.
- A contract is marked completed when the stage is Delivered and all three payments are paid.
- Photos can be uploaded only by the contract's farmer, and viewed only by that farmer, that merchant, or the admin.

These rules live in the server's controllers and middleware, never only in the interface. A rule that is only hidden by a disabled button can be bypassed by anyone who sends the request directly, so every rule above is checked again on the server and answered with a clear error message when it is broken.

3.2 Non-Functional Requirements

Category	Requirement
Usability	Mobile-first layout, base font of 18 px or more, buttons at least 48 px tall, icon plus label on every action
Localisation	English, Tamil, Hindi, Telugu and Malayalam, with English as fallback
Security	Hashed passwords, JWT authentication, role and ownership checks, input validation, rate limiting, request sanitisation, upload type checking
Performance	Client-side image compression to 1280 px at JPEG quality 0.8 to suit slow networks
Reliability	Offline queue for photo uploads, with automatic retry when the network returns
Auditability	Audit log entries for signing, stage changes, payments, disputes, verification and photo events
Portability	Storage layer behind an interface so cloud storage can replace local disk through an environment variable

Discussion. Usability is treated as a requirement and not a decoration. A farmer using a low-cost phone outdoors needs large touch targets and readable text, so the minimum font size and button height are fixed in the style sheet. Performance is expressed in terms of data size because mobile data in rural areas is slow and costly: a 4 MB photo from a modern phone camera is reduced to a few hundred kilobytes before it leaves the device. Reliability is expressed as the ability to continue working without a network; a farmer standing in a field with no signal can still take the photo, and the app sends it later.

3.3 Actors and Use Cases

Farmer. Registers, waits for approval, views contracts offered by merchants, signs a contract, advances the crop stage, uploads field photos, views payment status, raises a dispute and reads notifications.

Merchant (Buyer). Registers, waits for approval, creates a contract for a named farmer, signs it, views the stage and photo gallery, releases each payment milestone in order, raises a dispute and reads notifications.

Admin. Logs in with a seeded administrator account, verifies new users, views every contract and dispute, resolves disputes with a note and removes inappropriate photos. **System.** Writes audit entries, creates notifications and the three payment records when a contract becomes active.

Use case	Primary	Pre-condition	Result
	actor		
Register	Farmer or Merchant	Mobile number not already used	Account created, verification pending
Verify user	Admin	User exists and is unverified	User can create or sign contracts
Create contract	Merchant	Merchant verified, farmer exists	Contract in awaiting_signature status
Sign contract	Farmer or Merchant	Caller is a party and verified	Signature time stored; contract active when both have signed
Advance stage	Farmer	Contract active, stage below Delivered	Stage increased by one; update recorded
Upload photo	Farmer	Contract active	Photo stored, merchant notified
Release payment	Merchant	Earlier milestones paid, stage condition met	Payment marked paid, invoice number stored
Raise dispute	Farmer or Merchant	Caller is a party	Dispute opened, admin notified
Resolve dispute	Admin	Dispute open	Dispute resolved with note

3.4 Software and Hardware Requirements

Software requirements

Component	Technology
Front end	React with Vite, React Router, Axios, react-i18next
Back end	Node.js 18 or later, Express.js
Database	MongoDB Atlas (free tier) with Mongoose
Authentication	JSON Web Tokens, bcrypt
File upload	multer, local disk storage
Security packages	helmet, cors, express-rate-limit, express-mongo-sanitize
Validation	zod or express-validator
Testing	Jest and Supertest
Deployment	Render (server), Vercel (client), GitHub (source)
Development tools	Visual Studio Code, PowerShell, Git, Google Chrome
Operating system	Windows 10 or 11 for development; any modern OS for use

Hardware requirements

Item	Development machine	End-user device
Processor	Dual-core 2 GHz or better	Entry-level Android phone
Memory	8 GB RAM recommended	2 GB RAM
Storage	2 GB free for tools and dependencies	Small, since the app runs in the browser
Camera	Not required	Rear camera for field photos
Network	Broadband for installing packages and using Atlas	Mobile data; slow networks are supported

The camera feature works only on HTTPS or on localhost, which the deployed Vercel site satisfies.

3.5 Technology Justification

React with Vite. React's component model suits an application made of repeated pieces such as contract cards, stage steppers and photo tiles. Vite gives fast start-up and hot reload during development and produces a small optimised bundle for deployment, which matters on slow mobile networks.

Node.js and Express. Using JavaScript on both sides lets one developer write the client and the server and share naming and validation rules. Express is minimal, so the middleware chain (security, authentication, roles, validation, errors) is explicit and easy to explain.

MongoDB with Mongoose. A contract naturally holds nested and varying information such as the signature times and photo metadata. A document database stores such records without rigid joins, while Mongoose adds schemas, validation and references so that the data stays consistent.

JWT and bcrypt. A stateless token allows the server to be restarted or scaled without a session store. bcrypt is deliberately slow and salted, so a stolen database of password hashes cannot be cracked quickly.

react-i18next. It separates every visible string from the code so that a new language is only a new translation file, with English as the fallback when a key is missing.

CHAPTER 4

SYSTEM DESIGN

4.1 System Architecture

The system follows a three-tier architecture. The browser talks only to the Express API, and only the API talks to the database and the file store.

- **Presentation tier (React client, hosted on Vercel).** Pages, components, language and authentication context, an Axios client, the camera and compression code, and the offline upload queue.
- **Application tier (Express API, hosted on Render).** Routes call controllers. Middleware runs first for security headers, rate limiting, request sanitisation, JWT authentication, role checks, upload handling, validation and central error handling. Services hold payment, notification and audit logic.
- **Data tier.** MongoDB Atlas stores users, contracts, crop updates, photo metadata, payments, disputes, notifications and audit logs. Photo files are stored on the server disk behind a storage interface and are served only through an authenticated route.

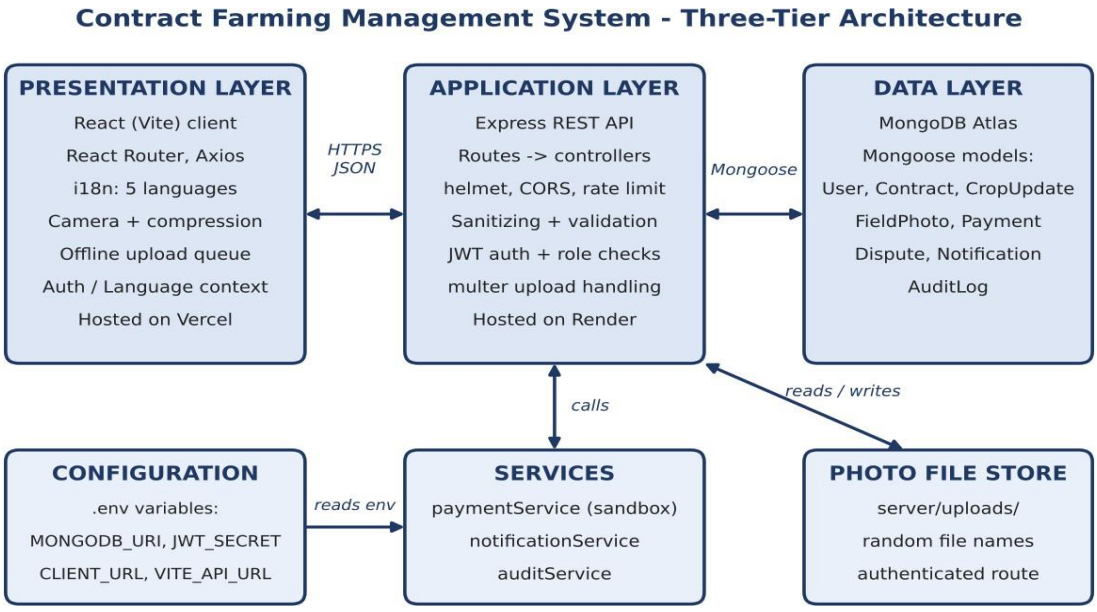


Figure 4.1: System architecture of the Contract Farming Management System

Request life cycle. A request leaves the React client through the Axios instance, which adds the Authorization header from the stored token. On the server, helmet sets security headers, the CORS rule checks the origin, the rate limiter counts the request, the body parser reads the JSON, and the sanitiser strips MongoDB operators. The authentication middleware verifies the token and loads the user. A role middleware may then check the role. Validation checks the body. Finally the controller runs, calls a service

if needed, and sends a JSON response. Any thrown error is caught by a central error handler, which returns a consistent message shape.

Why layers. Separating routes, controllers, services and models keeps each file small and testable. Routes only map a URL and method to a controller and its middleware. Controllers hold the business rules. Services wrap side effects such as payment, notification and audit logging. Models describe the data. Because the payment provider is hidden inside a service, replacing the sandbox with a real gateway changes one file.

4.2 Module Description

Module	Purpose	Main users
Welcome and language	First screen with language buttons, a five-step guide and a Listen button that reads each step aloud	All
Authentication	Registration, login, JWT issue, current user, language preference	All
User verification	Admin approves new users before they can create or sign contracts	Admin
Contracts	Create, list, view and sign contracts; status moves from awaiting signature to active to completed	Merchant, Farmer
Crop progress	Stage stepper and stage history for each contract	Farmer, Merchant
Field photos	Camera capture, compression, offline queue, upload, authenticated serving, gallery and viewer	Farmer, Merchant
Payments	Three ordered milestones, sandbox provider, invoice numbers	Merchant, Farmer
Disputes	Raise a problem, list, resolve with a note	All
Notifications	In-app messages and unread badge	All
Dashboard	Role-specific counts, totals and recent activity	All
Admin console	Verify users, view all contracts and disputes, remove photos	Admin
Audit log	Records of sensitive actions	System

Welcome and language module. Before login, a visitor sees a screen with one large button per language and a short illustrated guide. Each of the five steps can be read aloud with the Listen button. The chosen language is stored in the browser and later in the

user's profile, so the interface opens in the same language on every device.

Authentication module. It validates the registration form, rejects a duplicate mobile number, hashes the password and stores the user. On login it compares the password with the hash and issues a token that expires in seven days. A current-user endpoint lets the client restore the session after a page refresh.

Contract module. It creates contracts with a unique readable identifier such as CF-1042, enforces who may sign, and moves the status from awaiting signature to active when both signatures exist.

Crop progress module. It shows the four stages as a stepper. Only the farmer sees the button that advances to the next stage, and the server refuses any other caller. Each change is stored as a CropUpdate so that the history can be shown as a timeline.

Field photo module. It combines camera capture, compression, upload and viewing. The client reduces the image, the server verifies it, and the gallery shows photos grouped by stage with a full-screen viewer. A photo that the merchant opens is marked as seen.

Dispute module. It lets a party describe a problem against a contract. The admin sees open disputes, reads the contract history and closes the dispute with a note, which both parties can then read.

Notification and dashboard modules. Notifications are short messages stored per user with an unread flag and a link to the relevant contract. The dashboard summarises counts of contracts by status, payment totals and recent activity according to the role.

4.3 Database Design

The database is named contract_farming. Each collection maps to one Mongoose model.

Collection	Key fields
User	name, phone (unique), passwordHash, role (farmer, buyer, admin), language, village or company, verified, documents, createdAt
Contract	contractId (for example CF-1042), farmer, buyer, crop, quantityQuintals, pricePerQuintal, grade (A, B, C), deliveryDate, signedByFarmer, signedByBuyer, stage (0 to 3), status
CropUpdate	contract, stage, note, createdBy, createdAt
FieldPhoto	contract, stage, note, filePath, mimeType, size, location (lat, lng), uploadedBy, takenAt, seenByBuyer
Payment	contract, milestone (0 to 2), percent (30, 40, 30), amount, status (pending, paid), paidAt, invoiceNo

Design decisions. The phone number carries a unique index, which is what makes duplicate registration impossible even if two requests arrive at the same moment. Contracts keep references to the farmer and the buyer instead of copies of their details, so a change to a user's profile is reflected everywhere. Payments are stored as three separate documents per contract and not as an array inside the contract; this keeps each payment independently queryable and lets the invoice number be unique. Photo metadata is stored in MongoDB but the image bytes are stored in the file store, which keeps documents small. The amount of each payment is computed once, when the contract becomes active, as the percentage of quantity times price per quintal, so later display does not depend on recalculation.

Indexes. The system relies on a unique index on User.phone, an index on Contract.farmer and Contract.buyer for the role-scoped list queries, an index on FieldPhoto.contract for the gallery, and an index on Notification.user together with read for the unread badge.

4.4 Class Diagram

The class diagram shows the main Mongoose models. It is a conceptual data model, not a method-level UML diagram. The relationships are:

- One User (farmer) has many Contracts; one User (buyer) has many Contracts. Each Contract has exactly one farmer and one buyer.
- One Contract has many CropUpdates, many FieldPhotos, exactly three Payments and zero or more Disputes.
- A FieldPhoto, CropUpdate and Dispute each refer to the User who created them.
- A User has many Notifications. An AuditLog entry refers to the acting User.

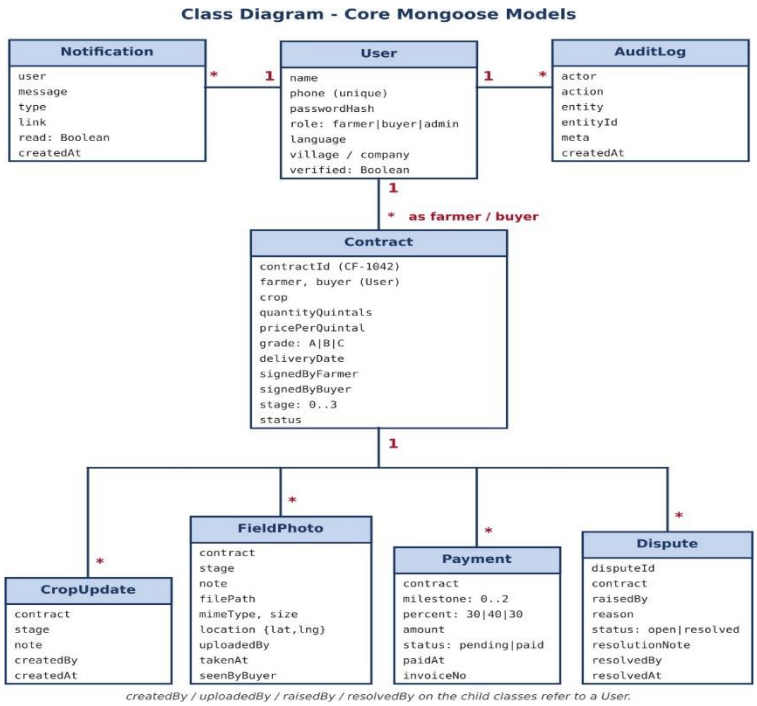


Figure 4.2: Class diagram of the core data models

4.5 Sequence Diagrams

Figure 4.3 shows the sequence for creating and signing a contract. Figure 4.4 shows the sequence for uploading a field photo and releasing a milestone payment.

Flow A: signing a contract.

- The merchant submits the new contract form. The client sends POST /api/contracts.
- The API checks the token, the buyer role and the verified flag, validates the body, and stores the contract with status awaiting_signature.
- The merchant confirms with their password. The client sends PATCH /api/contracts/:id/sign. The API checks the password hash and records signedByBuyer with a time.
- The farmer opens the contract and signs the same way. The API records signedByFarmer.
- When both signatures exist, the API sets the status to active, creates the three Payment records, writes an audit entry and notifies both users.

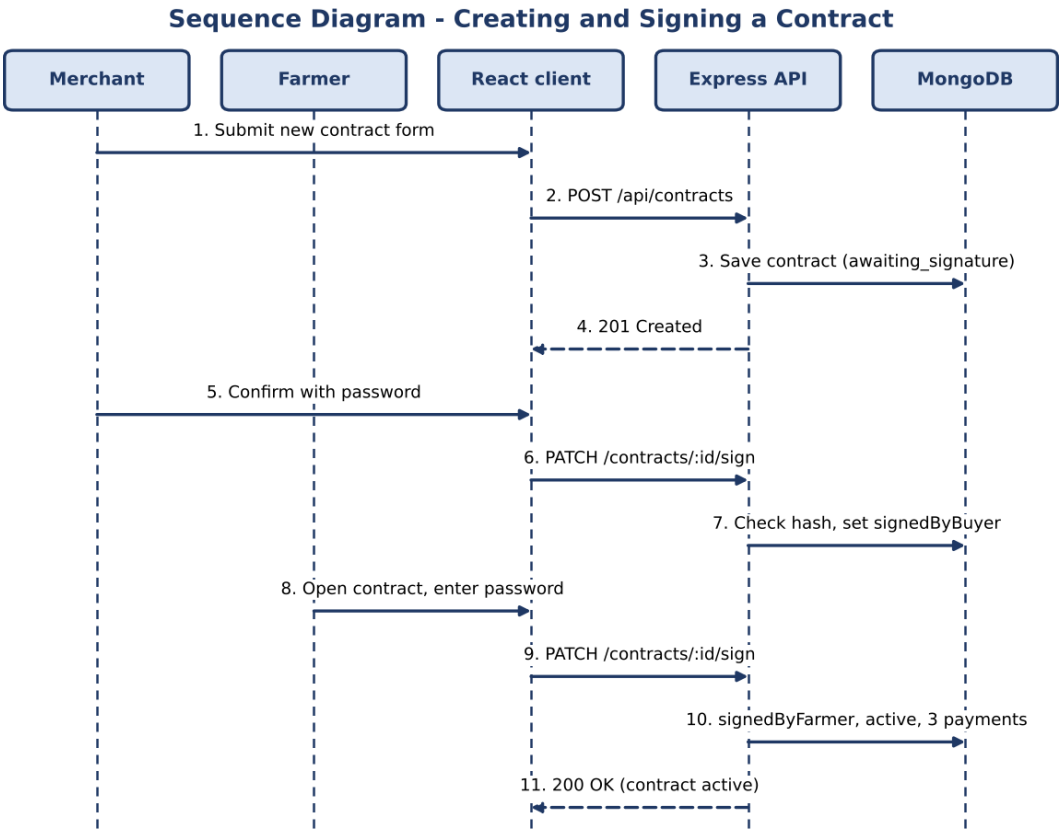


Figure 4.3: Sequence diagram for creating and signing a contract

Flow B: photo upload and payment.

- The farmer takes a photo. The client compresses it to at most 1280 px and sends POST /api/contracts/:id/photos as multipart data. If offline, the photo waits in the queue and is sent when the network returns.
- The API checks that the caller is this contract's farmer and the contract is active, verifies the real file type and size, saves the file under a random name, and stores the metadata.
- The API creates a notification for the merchant and an audit entry.
- When the stage reaches Harvest, the merchant sends POST /api/contracts/:id/payments/1/release. The API checks the order and the stage condition, calls the sandbox payment service, and stores the invoice number.

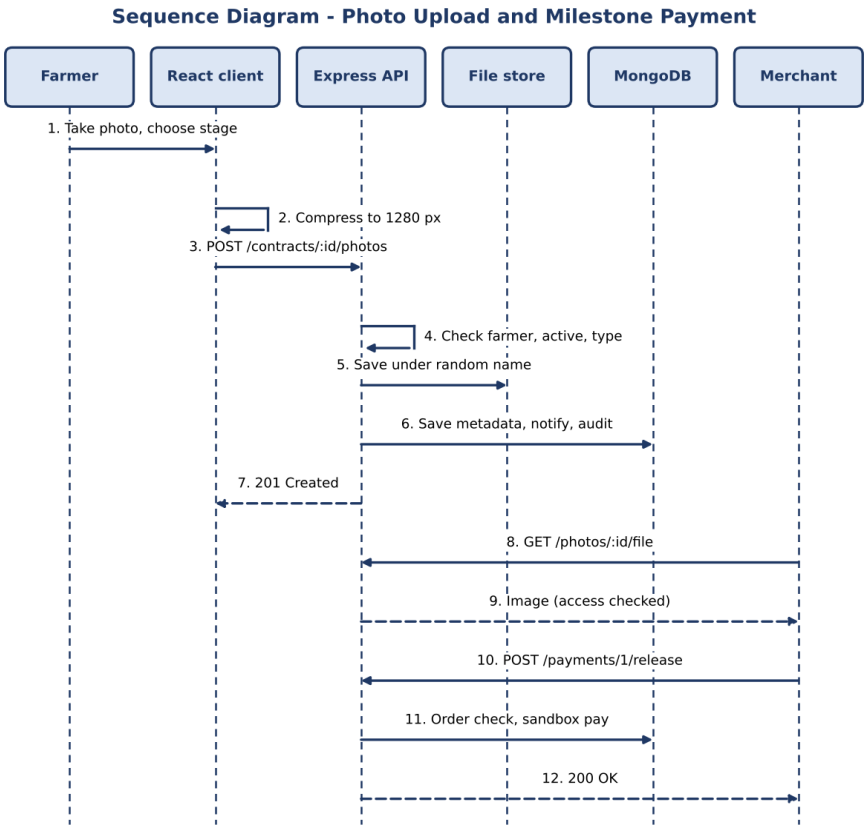


Figure 4.4: Sequence diagram for photo upload and milestone payment

4.6 State Models

Contract status. A contract starts as awaiting_signature. It becomes active when both parties have signed. It becomes completed when the stage is Delivered and all three payments are paid. No transition goes backwards, which keeps the history simple and trustworthy.

Crop stage. The stage is a number from 0 to 3: Sowing (0), Growth (1), Harvest (2) and Delivered (3). The only allowed change is stage plus one. The server rejects any attempt to skip a stage or to go back.

Payment milestone. Each payment is pending until released and then paid. Milestone 0 may be released once the contract is active, milestone 1 once the stage is Harvest or later, and milestone 2 once the stage is Delivered, always after the previous milestone is paid. **Dispute.** A dispute is open until the admin resolves it with a note, after which it is resolved and read-only.

Milestone	Share	Condition for release	Typical meaning
0	30%	Contract active (both signed)	Advance to help the farmer buy inputs
1	40%	Stage is Harvest	Payment when the crop is cut and ready
2	30%	Stage is Delivered	Final payment after the crop reaches the buyer

4.7 REST API Design

Method and path	Purpose	Who may call
POST /api/auth/register	Create an account	Anyone
POST /api/auth/login	Log in and receive a token	Anyone
GET /api/auth/me	Return the current user	Any logged-in user
PATCH /api/auth/language	Save the language preference	Any logged-in user
GET /api/users/pending	List users awaiting approval	Admin
PATCH /api/users/:id/verify	Approve a user	Admin
POST /api/contracts	Create a contract	Verified merchant
GET /api/contracts	List contracts scoped to the caller	Any logged-in user
GET /api/contracts/:id	Read one contract	A party or admin
PATCH /api/contracts/:id/sign	Sign with password	A verified party
PATCH /api/contracts/:id/stage	Advance the stage	Contract's farmer
POST /api/contracts/:id/photos	Upload a field photo	Contract's farmer
GET /api/contracts/:id/photos	List photo metadata	A party or admin
GET /api/photos/:photoId/file	Return the image bytes	A party or admin
DELETE /api/photos/:photoId	Remove a photo	Admin
GET /api/contracts/:id/payments	List the three payments	A party or admin
POST /api/contracts/:id/payments/:n/release	Release milestone n	Contract's merchant
POST /api/disputes	Raise a dispute	A party
PATCH /api/disputes/:id/resolve	Resolve with a note	Admin
GET /api/notifications	List the caller's notifications	Any logged-in user

All responses follow one shape. A success returns an object with success set to true and a data field. A failure returns success set to false with a message and, for validation failures, an errors array of field and message pairs. A single shape lets the client show errors beside the right input without special cases.

4.8 User Interface Design

The interface follows five principles. First, **one task per screen**: the contract screen shows the contract, the stage screen shows the stepper, and so on, instead of one crowded page. Second, **large touch targets**: all buttons are at least 48 px tall and the base font is at least 18 px. Third, **icon plus label**: every action pairs a clear icon with a word so that a user who reads slowly can still recognise it. Fourth, **colour with meaning but never colour alone**: statuses use colour and a text label. Fifth, **language first**: the very first screen asks for a language and offers spoken help.

The farmer's navigation contains My Contracts, Crop Progress, Photos, Payments, Problems and Notifications. The merchant's navigation contains New Contract, My Contracts, Photos, Payments, Problems and Notifications. The admin's navigation contains Verify Users, All Contracts, Problems and Photos.

4.9 Security Design

Security is designed in layers so that one failure does not expose the system.

Threat	Control
Stolen password database	Passwords stored only as bcrypt hashes
Forged or tampered token	JWT signed with a secret kept in an environment variable
Calling an API without the right role	Role middleware plus ownership checks in each controller
Reading another farmer's contract or photos	Ownership check before any contract or photo is returned

4.10 User Interface Design

The interface follows five principles. First, **one task per screen**: the contract screen shows the contract, the stage screen shows the stepper, and so on, instead of one crowded page. Second, **large touch targets**: all buttons are at least 48 px tall and the base font is at least 18 px. Third, **icon plus label**: every action pairs a clear icon with a word so that a user who reads slowly can still recognise it. Fourth, **colour with meaning but never colour alone**: statuses use colour and a text label. Fifth, **language first**: the very first screen asks for a language and offers spoken help.

The farmer's navigation contains My Contracts, Crop Progress, Photos, Payments, Problems and Notifications. The merchant's navigation contains New Contract, My Contracts, Photos, Payments, Problems and Notifications. The admin's navigation contains Verify Users, All Contracts, Problems and Photos.

4.11 Security Design

Security is designed in layers so that one failure does not expose the system.

Threat	**Control**
Stolen password database	Passwords stored only as bcrypt hashes
Forged or tampered token	JWT signed with a secret kept in an environment variable
Calling an API without the right role	Role middleware plus ownership checks in each controller
Reading another farmer's contract or photos	Ownership check before any contract or photo is returned
NoSQL operator injection	express-mongo-sanitize removes operator keys from input
Brute-force login	Stricter rate limit on login and registration
Malicious upload	5 MB limit, real file signature check, random file name, no direct folder exposure
Cross-origin abuse	CORS restricted to the client address
Missing security headers	helmet
Leaked secrets	.env file excluded from source control, only variable names documented

CHAPTER 5

IMPLEMENTATION

5.1 Implementation Overview

The project is a monorepo with a client folder and a server folder. The server is organised as routes, controllers, models, middleware, services and config. The client is organised as pages, components, hooks, api, i18n and context.

The work was done in seven phases: (1) scaffold, database connection, models, authentication and role middleware; (2) contract, signing, stage, payment and dispute APIs with a seed script; (3) photo upload with multer, authenticated file serving and notifications; (4) React shell, five-language support, the welcome guide, login and registration; (5) dashboard, contract, payment, dispute and admin screens wired to the API; (6) camera capture, compression, offline queue and the merchant gallery; (7) tests, security review and documentation.

The database is hosted on MongoDB Atlas. A seed script (npm run seed) creates the demo admin, two farmers, one merchant, three sample contracts, sample photos and one open dispute. During development, npm run dev starts the Express server and the Vite client together. For deployment, the server runs on Render and the client on Vercel. Connection details are kept in environment variables (MONGODB_URI, JWT_SECRET, CLIENT_URL, VITE_API_URL) and are not stored in the source code.

5.2 Development Phases in Detail

Phase	Work done	Result
1	Folder scaffold, Atlas connection, User model, register and login, auth and role middleware	A user can register, log in and call a protected route
2	Contract, CropUpdate, Payment and Dispute models and controllers; seed script	The full business flow works from an API client
3	multer upload, signature check, authenticated file route, notification and audit services	Photos can be uploaded and viewed with access control
4	React shell, router, auth context, i18n with five languages, welcome guide with Listen button	A visitor can choose a language and register
5	Dashboard, contract list and detail, sign dialog, stage stepper, payments, disputes, admin console	Every role can complete its tasks in the browser
6	Camera capture, compression, IndexedDB queue, merchant gallery and viewer	Photo evidence works on a phone, including offline
7	Jest and Supertest tests, security review, deployment, documentation	A tested and deployed prototype

5.3 Key Functions (Code Explanation)

The snippets below are representative of the logic in each module.

1. Registration and password hashing. The registration controller validates the input, rejects a mobile number that already exists, hashes the password with bcrypt and stores the user with the chosen role (farmer or buyer). The plain password is never stored. A cost factor of 10 means bcrypt performs 2^10 rounds, which is slow enough to resist guessing but fast enough that a login feels instant.

```
const exists = await User.findOne({ phone });
if (exists) return res.status(409).json({ success: false, message: 'Mobile number
    already registered' });
const passwordHash = await bcrypt.hash(password, 10);
const user = await User.create({ name, phone, passwordHash, role, village, company });
```

2. Login and JWT. The login controller finds the user by phone, compares the password with the stored hash, and returns a signed token that carries the user's id and role. The same error message is returned whether the phone number or the password is wrong, so an attacker cannot learn which mobile numbers are registered.

```
const ok = await bcrypt.compare(password, user.passwordHash);
if (!ok) return res.status(401).json({ success: false, message: 'Invalid credentials'
    });
const token = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_SECRET, {
    expiresIn: '7d' });
```

3. Authentication and role middleware. Every protected route first verifies the token and loads the user. A second middleware then checks the role. This is why hiding a button in the interface is never the only protection: the server refuses the request even if someone calls the API directly.

```
export const protect = async (req, res, next) => {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ success: false, message: 'Login required'
      });
  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    req.user = await User.findById(payload.id).select('-passwordHash');
    if (!req.user) return res.status(401).json({ success: false, message: 'User not
        found' });
    next();
  } catch (err) {
    return res.status(401).json({ success: false, message: 'Invalid or expired token'
        });
  }
};
export const requireRole = (...roles) => (req, res, next) =>
  roles.includes(req.user.role)
    ? next()
```

4. Contract creation. The create function checks that the caller is a verified merchant, that the named farmer exists and has the farmer role, and then stores the contract with a readable identifier. The quantity and price are validated as positive numbers before the controller runs.

```
const farmerUser = await User.findOne({ _id: farmer, role: 'farmer' });
if (!farmerUser) throw new AppError('Farmer not found', 404);
const contractId = 'CF-' + (1000 + (await Contract.countDocuments()) + 1);
const contract = await Contract.create({
  contractId, farmer, buyer: req.user._id, crop,
  quantityQuintals, pricePerQuintal, grade, deliveryDate,
  stage: 0, status: 'awaiting_signature'
});
await audit.log(req.user._id, 'CONTRACT_CREATED', 'Contract', contract._id);
```

5. Contract signing. The sign function checks that the caller is the contract's farmer or buyer and that the user is verified. It re-checks the password, records the signature time for that party, and moves the contract to active only when both signatures exist. At that moment it also creates the three payment records with their amounts.

```
const isFarmer = contract.farmer.equals(req.user._id);
const isBuyer = contract.buyer.equals(req.user._id);
if (!isFarmer && !isBuyer) throw new AppError('Not your contract', 403);
const fresh = await User.findById(req.user._id);
if (!(await bcrypt.compare(req.body.password, fresh.passwordHash)))
  throw new AppError('Wrong password', 401);
if (isFarmer) contract.signedByFarmer = new Date();
if (isBuyer) contract.signedByBuyer = new Date();
if (contract.signedByFarmer && contract.signedByBuyer) {
  contract.status = 'active';
  const total = contract.quantityQuintals * contract.pricePerQuintal;
  const shares = [30, 40, 30];
  await Payment.insertMany(shares.map((percent, milestone) => ({
    contract: contract._id, milestone, percent,
    amount: Math.round(total * percent / 100), status: 'pending'
  })));
}
await contract.save();
```

6. Stage advance. The stage function allows only the contract's farmer, only on an active contract, and only to the next stage (current stage plus one). Each change creates a CropUpdate, a notification for the merchant and an audit entry.

```
if (!contract.farmer.equals(req.user._id)) throw new AppError('Only the farmer can
    update the stage', 403);
if (contract.status !== 'active') throw new AppError('Contract is not active', 400);
if (req.body.stage !== contract.stage + 1) throw new AppError('Stage can move one
    step at a time', 400);
contract.stage = req.body.stage;
await contract.save();
await CropUpdate.create({ contract: contract._id, stage: contract.stage, note:
    req.body.note, createdBy: req.user._id });
await notify(contract.buyer, 'Crop stage updated on ' + contract.contractId, 'stage',
    '/contracts/' + contract._id);
```

7. Milestone payments. The release function checks that the caller is the contract's buyer, that all earlier milestones are paid, and that the stage condition holds: milestone 0 after signing, milestone 1 at Harvest, milestone 2 at Delivered. It then calls the payment service, which in this version is a sandbox provider that marks the payment paid and generates an invoice number. When the stage is Delivered and all three payments are paid, the contract becomes completed.

```
const needStage = [0, 2, 3][milestone];
if (contract.stage < needStage) throw new AppError('Stage condition not met', 400);
if (milestone > 0 && payments[milestone - 1].status !== 'paid')
  throw new AppError('Release earlier milestones first', 400);
const result = await paymentService.charge({ amount, contractId });
payment.status = 'paid';
payment.paidAt = new Date();
payment.invoiceNo = result.invoiceNo;
await payment.save();
if (contract.stage === 3 && payments.every(p => p.status === 'paid')) {
  contract.status = 'completed';
  await contract.save();
}
```

The payment service is deliberately small. It exposes one function, charge, which returns an invoice number. A real gateway would implement the same function by creating an order, waiting for a webhook and returning the gateway's reference.

```
export const paymentService = {
  async charge({ amount, contractId }) {
    return { ok: true, invoiceNo: 'INV-' + contractId + '-' + Date.now() };
  }
};
```

8. Photo upload and access control. The upload route uses multer with a 5 MB limit. The server reads the file's real signature (its first bytes) to confirm it is JPEG, PNG or WebP, stores it under a random name, and saves the metadata. The file route does not expose the upload folder directly. It first checks that the requester is the contract's farmer, the contract's merchant or the admin.

```
const upload = multer({ storage: multer.memoryStorage(), limits: { fileSize: 5 * 1024
    * 1024 } });
function detectType(buf) {
  if (buf[0] === 0xff && buf[1] === 0xd8) return 'image/jpeg';
  if (buf.slice(0, 4).toString('hex') === '89504e47') return 'image/png';
  if (buf.slice(0, 4).toString() === 'RIFF' && buf.slice(8, 12).toString() ===
```

```
      'WEBP') return 'image/webp';
  return null;
}
const type = detectType(req.file.buffer);
if (!type) throw new AppError('Only JPEG, PNG or WebP images are allowed', 400);
const name = crypto.randomBytes(16).toString('hex') + '.' + type.split('/')[1];
await storage.save(name, req.file.buffer);
```

9. Camera capture and compression (client). The client asks for the rear camera with getUserMedia, draws the captured frame to a canvas, scales it so the longest side is at most 1280 px, and exports JPEG at quality 0.8. If the camera is unavailable or permission is denied, a file input with capture set to environment is used instead.

```
const stream = await navigator.mediaDevices.getUserMedia({ video: { facingMode:
    'environment' } });
video.srcObject = stream;
const scale = Math.min(1, 1280 / Math.max(video.videoWidth, video.videoHeight));
canvas.width = Math.round(video.videoWidth * scale);
canvas.height = Math.round(video.videoHeight * scale);
canvas.getContext('2d').drawImage(video, 0, 0, canvas.width, canvas.height);
const blob = await new Promise(r => canvas.toBlob(r, 'image/jpeg', 0.8));
```

10. Location tagging. When the farmer grants permission, the browser's Geolocation API supplies latitude and longitude, which are sent with the photo. If permission is refused the photo is still uploaded without coordinates, because a photo without a location is better than no photo.

```
navigator.geolocation.getCurrentPosition(
  pos => setLocation({ lat: pos.coords.latitude, lng: pos.coords.longitude }),
  () => setLocation(null),
  { enableHighAccuracy: true, timeout: 8000 }
```

11. Offline upload queue. When the network is unavailable, the photo and its metadata are saved in IndexedDB and shown with a waiting-to-upload badge. The client listens for the browser's online event and sends the queued items in order. An item is removed from the queue only after the server confirms the upload, so a failure leaves it in place for the next attempt.

```
window.addEventListener('online', async () => {
  const items = await queue.getAll();
  for (const item of items) {
    try {
      await api.post('/contracts/' + item.contractId + '/photos', toFormData(item));
      await queue.remove(item.id);
    } catch (err) {
      break;
    }
  }
});
```

12. Multilingual interface and spoken guide. All text comes from translation files for the five languages through react-i18next. The chosen language is saved in localStorage and, after login, in the user's profile. The Listen button uses the Web Speech API and is hidden when the browser has no voice for the chosen language.

```
function speak(text, lang) {
  const voices = window.speechSynthesis.getVoices();
  if (!voices.some(v => v.lang.startsWith(lang))) return false;
  const u = new SpeechSynthesisUtterance(text);
```

```
  u.lang = lang;
  window.speechSynthesis.speak(u);
  return true;
}
```

13. Audit and notification services. Both services are thin wrappers around a model create call, but keeping them in one place guarantees that every sensitive action records the same fields. The audit log stores the actor, the action name, the entity type and identifier, and a small meta object.

```
export const audit = {
  log: (actor, action, entity, entityId, meta = {}) =>
    AuditLog.create({ actor, action, entity, entityId, meta })
};
export const notify = (user, message, type, link) =>
  Notification.create({ user, message, type, link, read: false });
```

14. Security configuration. The server uses helmet for headers, cors restricted to the client address, express-rate-limit with a stricter limit on login and registration, and express-mongo-sanitize to block operator injection. Validation errors are returned as a list of field and message pairs, which the client shows next to the matching field.

```
app.use(helmet());
app.use(cors({ origin: process.env.CLIENT_URL }));
app.use(express.json({ limit: '100kb' }));
app.use(mongoSanitize());
app.use('/api', rateLimit({ windowMs: 15 * 60 * 1000, max: 300 }));
app.use('/api/auth', rateLimit({ windowMs: 15 * 60 * 1000, max: 20 }));
```

5.4 Deployment

The client is built with Vite and deployed to Vercel, which serves the static files over HTTPS; HTTPS is required for the camera feature. The server is deployed to Render as a Node web service whose start command runs server.js. The environment variables are entered in each platform's settings page and not in the repository. After deployment, the client's VITE_API_URL points to the Render address and the server's CLIENT_URL points to the Vercel address, so that CORS accepts only the real client.

One limitation of the free hosting tier is that the server may sleep when idle and take some seconds to wake on the first request. The client shows a loading message during this wait. Another is that files stored on the server's disk may be lost when the service is rebuilt, which is why the storage interface was designed to allow a switch to cloud storage by an environment variable.

CHAPTER 6

TESTING AND RESULTS

6.1 Testing Strategy

Testing was carried out at four levels. **Unit tests** check single functions such as the file type detector and the milestone condition. **API tests** use Jest with Supertest to send real HTTP requests to the Express app and check status codes and bodies. **Manual end-to-end tests** follow the full flow in the browser with the seeded accounts. **Usability and security checks** examine mobile layout, language switching, access control and upload handling. **Test environment.** Node.js 18, MongoDB Atlas test database seeded with the demo data, Google Chrome on a Windows laptop, and Chrome on an Android phone for the camera and offline tests.

6.2 Functional Test Cases

ID	Scenario	Input	Expected result	Result
T1	Register a new farmer	Valid name, 10-digit phone, password, village	201 Created, user unverified	Pass
T2	Register with existing phone	Phone already stored	409 and message	Pass
T3	Register with a 9-digit phone	Phone with 9 digits	400 with field error on phone	Pass
T4	Login with correct details	Correct phone and password	200 and a token	Pass
T5	Login with wrong password	Wrong password	401 Invalid credentials	Pass
T6	Access protected route without token	No Authorization header	401 Login required	Pass
T7	Unverified merchant creates contract	Valid body, unverified user	Request refused	Pass
T8	Verified merchant creates contract	Valid body	201, status awaiting_signature	Pass
T9	Farmer tries to create contract	Farmer token	403 Not allowed	Pass
T10	Sign with wrong password	Wrong password	401 Wrong password	Pass
T11	Both parties sign	Correct passwords	Status active, 3 payments created	Pass

ID	Scenario	Input	Expected result	Result
T12	Stranger signs a contract	Token of an unrelated user	403 Not your contract	Pass
T13	Advance stage on inactive contract	Contract awaiting signature	400 Contract is not active	P ass
T14	Skip a stage	Stage 0 to stage 2	400 one step at a time	Pass
T15	Merchant advances stage	Merchant token	403 Only the farmer	Pass
T16	Farmer advances stage correctly	Stage 0 to 1	200, CropUpdate and notification created	Pass
T17	Release milestone 1 before Harvest	Stage is Growth	400 Stage condition not met	Pass
T18	Release milestone 2 before milestone 1	Milestone 1 pending	400 Release earlier milestones first	Pass
T19	Release milestone 0 after signing	Merchant token	200, invoice number stored	Pass
T20	Contract completion	Stage 3 and all paid	Status completed	Pass
T21	Upload a valid JPEG	Photo under 5 MB	201, merchant notified	Pass
T22	Upload a renamed text file as .jpg	Text bytes, .jpg name	400 Only JPEG, PNG or WebP	Pass
T23	Upload a file over 5 MB	6 MB image	413 or 400 size error	Pass
T24	Stranger views a photo	Unrelated user's token	403	Pass
T25	Farmer deletes a photo	Farmer token	403 Not allowed	Pass
T26	Admin deletes a photo	Admin token	200, audit entry written	Pass
T27	Raise and resolve a dispute	Farmer raises, admin resolves with note	Dispute resolved, both parties notified	Pass
T28	NoSQL injection in login	Phone as an object with an operator key	Operator removed, login fails	Pass

6.3 Non-Functional Test Results

Area	Test	Observation
Image size	4 MB phone photo uploaded after compression	Reduced to a few hundred kilobytes at 1280 px and quality 0.8
Offline behaviour	Network switched off, three photos taken, network restored	Three items shown as waiting, all sent in order after reconnect
Language	Interface switched among all five languages	All labels translated; missing keys fall back to English
Spoken guide	Listen button on a phone with Tamil voice installed	Guide read aloud; button hidden where no voice exists
Mobile layout	360 px wide viewport	No horizontal scrolling; buttons at least 48 px tall
Rate limit	25 login attempts in a short time	Requests after the limit were refused with 429
Camera fallback	Permission denied	File input with capture used instead

6.4 Issues Found and Corrected

During integration testing, mismatches between the client and the server were found and corrected. The client sent the role name merchant while the server expected buyer; the validation error in the appendix (Example 4) shows the exact message the server returns for such a mismatch. Some field names differed between the form and the schema, for instance the quantity field. A route used PUT in the client while the server defined PATCH. A photo URL used in the gallery pointed at the upload folder directly instead of the authenticated file route. Each issue was corrected by treating the server's validation rules as the single source of truth and updating the client to match.

6.5 Discussion of Results

All functional test cases passed after the corrections. The most important result is that every business rule is enforced on the server: the tests that call the API directly with the wrong role, the wrong order or the wrong user were all refused. This confirms that the interface is not the only line of defence.

The offline queue and compression gave the intended benefit on a mobile network: photos did not block the farmer's work, and the amount of data sent was greatly reduced. The five-language interface and the spoken guide were verified functionally, but their effect on real farmers has not been measured in this version; a field trial with actual farmers is listed as future work.

6.6 Limitations

- The digital signature is a password-confirmed record and is not a legally certified e-signature.
- Payment is simulated; no real money moves and there is no refund or webhook handling.
- Photo files are stored on server disk, which is not durable on free hosting.
- Location coordinates come from the device and can be disabled or altered by the user, so they strengthen the evidence but do not prove it.
- The Listen feature depends on voices installed on the user's device, which vary by phone.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 Conclusion

The Contract Farming Management System shows that the main steps of a farming agreement can be handled on one platform that a farmer can use from a phone. A contract is created and signed by both parties, the crop is tracked through four stages, field photos give the merchant evidence of progress, and payment is released in three ordered milestones. The admin can verify users and settle disputes, and every sensitive action leaves an audit record.

The design choices follow the needs of the users. The interface is available in five regional languages and reads its guide aloud. Photos are compressed and queued when the network fails. Access rules are checked on the server, so the application stays safe even if someone bypasses the screen. The sandbox payment service shows the full payment flow without moving real money, and its separate service layer means a real gateway can be added without changing the rest of the code.

During testing, mismatches between the client and the server (field names, role names and route methods) were found and corrected, which confirmed the value of treating the server's validation rules as the single source of truth.

The project also met its learning goals as a full stack exercise. It required designing a data model, building a secured REST API, writing a responsive and internationalised single-page client, handling files and device features in the browser, writing automated tests and deploying the two halves to separate hosting services. The result is a working prototype that can be demonstrated end to end with seeded data, and a clear path to a more complete product.

Contributions of the work. The work contributes (1) a contract model that links agreement, crop stage, photographic evidence and payment in one record; (2) a rule set in which payment can be released only in order and only when the crop stage supports it; (3) a farmer-oriented interface with regional languages, spoken help and offline photo queueing; and (4) a layered security design in which every rule is enforced on the server.

7.2 Future Enhancements

- **Real payments.** Replace the sandbox provider with a gateway such as Razorpay, with webhook confirmation and refunds.

- **Legally recognised signatures.** Integrate an Aadhaar-based eSign service or an OTP sent by SMS, so that signing carries legal weight.
- **Cloud file storage.** Move photos from local disk to Cloudinary or Amazon S3 through the existing storage interface.
- **Installable app and push alerts.** Convert the client to a Progressive Web App with push notifications.
- **Crop advice and photo analysis.** Add weather data and image-based crop health checks on the uploaded photos.
- **Price guidance.** Show recent mandi prices beside the contract price to help the farmer judge a fair rate.
- **Voice input and more languages.** Allow spoken entry for forms and add further regional languages.
- **Ratings and history.** Build a trust score for farmers and merchants from completed contracts and resolved disputes.
- **Reports.** Export contracts, payment receipts and dispute records as PDF.
- **Field trial.** Test the interface with real farmers and merchants, measure task completion time and errors, and revise the screens from the findings.
- **Contract templates.** Offer crop-specific templates that follow the structure of the model contract farming law, so that parties start from a fair default.

APPENDIX

SOURCE CODE

The code in this appendix is representative of the project files. Replace each block with the exact code from your own files before submission. Do not paste your .env values (database password, JWT secret) into the report.

A.1 Project Structure

```
FSP/
├── client/
│   ├── src/
│   │   ├── api/
│   │   ├── components/
│   │   ├── context/    (auth, language)
│   │   ├── hooks/
│   │   ├── i18n/       (en, ta, hi, te, ml)
│   │   ├── pages/
│   │   ├── App.jsx
│   │   └── main.jsx
│   ├── package.json
│   └── index.html
├── server/
│   ├── config/
│   ├── controllers/
│   ├── middleware/   (auth, role, upload, validate, error)
│   ├── models/
│   ├── routes/
│   ├── services/      (payment, notification, audit)
│   ├── uploads/
│   ├── .env.example
│   ├── package.json
│   └── server.js
└── package.json
```

A.2 Backend Package Installation

```
npm install express mongoose bcrypt jsonwebtoken cors helmet
npm install express-rate-limit express-mongo-sanitize multer dotenv zod
npm install --save-dev jest supertest nodemon
npm run install:all
npm run seed
npm run dev
```

A.3 Database Connection

server/config/db.js

```
import mongoose from 'mongoose';
export async function connectDB() {
  try {
    await mongoose.connect(process.env.MONGODB_URI, { dbName: 'contract_farming' });
    console.log('MongoDB connected');
  } catch (err) {
  console.error('MongoDB connection failed:', err.message);process.exit(1);}
}
```

A.4 Environment Variables

Variable names only, with the values blanked:

```
PORT=5000
MONGODB_URI=<your Atlas connection string>
JWT_SECRET=<a long random string>
CLIENT_URL=<your client address>
```

A.5 User Model

server/models/User.js

```
import mongoose from 'mongoose';
const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  phone: { type: String, required: true, unique: true, match: /^[0-9]{10}$/ },
  passwordHash: { type: String, required: true },
  role: { type: String, enum: ['farmer', 'buyer', 'admin'], required: true },
  language: { type: String, enum: ['en', 'ta', 'hi', 'te', 'ml'], default: 'en' },
  village: String,
  company: String,
  verified: { type: Boolean, default: false },
  documents: [String]
}, { timestamps: true });
export default mongoose.model('User', userSchema);
```

A.6 Password Hashing and Registration

server/controllers/authController.js

```
export const register = async (req, res, next) => {
  try {
    const { name, phone, password, role, village, company } = req.body;
    if (await User.findOne({ phone }))
      return res.status(409).json({ success: false, message: 'Mobile number already
          registered' });
    const passwordHash = await bcrypt.hash(password, 10);
    const user = await User.create({ name, phone, passwordHash, role, village,
        company });
    res.status(201).json({ success: true, data: { id: user._id, name: user.name,
        role: user.role } });
  } catch (err) {
    next(err);
  }
};
```

A.7 Login and JWT Generation

```
export const login = async (req, res, next) => {
  try {
    const { phone, password } = req.body;
    const user = await User.findOne({ phone });
    const ok = user && await bcrypt.compare(password, user.passwordHash);
    if (!ok) return res.status(401).json({ success: false, message: 'Invalid
        credentials' });
    const token = jwt.sign({ id: user._id, role: user.role }, process.env.JWT_SECRET,
        { expiresIn: '7d' });
    res.json({ success: true, data: { token, user: { id: user._id, name: user.name,
        role: user.role, language: user.language, verified: user.verified } } });
  } catch (err) {
    next(err);
  }
};
```

A.8 Authentication Middleware

server/middleware/auth.js and role.js: see the protect and requireRole functions in Chapter 5, Section 5.3, item 3.

A.9 Create API

server/routes/contractRoutes.js (excerpt)

```
router.post('/', protect, requireRole('buyer'), requireVerified,
    validate(contractSchema), createContract);
router.post('/:id/photos', protect, requireRole('farmer'), upload.single('photo'),
    uploadPhoto);
```

The createContract controller is shown in Chapter 5, item 4. The photo upload is a second create example and is shown in item 8.

A.10 Read API

The list function scopes the result by role so that a user never sees another user's contracts.

```
export const listContracts = async (req, res, next) => {
  try {
    const filter = req.user.role === 'admin' ? {}
      : req.user.role === 'farmer' ? { farmer: req.user._id }
      : { buyer: req.user._id };
    const contracts = await Contract.find(filter)
      .populate('farmer', 'name village')
      .populate('buyer', 'name company')
      .sort({ createdAt: -1 });
    res.json({ success: true, data: contracts });
  } catch (err) {
    next(err);
  }
};
export const getContract = async (req, res, next) => {
  try {
    const c = await Contract.findById(req.params.id).populate('farmer buyer', 'name');
    if (!c) throw new AppError('Contract not found', 404);
    const party = c.farmer._id.equals(req.user._id) ||
        c.buyer._id.equals(req.user._id);
    if (!party && req.user.role !== 'admin') throw new AppError('Not allowed', 403);
```

A.11 Update

The sign-contract and advance-stage controllers are shown in Chapter 5, items 5 and 6.

A.12 Delete

The delete function is admin-only and removes both the file and its metadata, then writes an audit entry.

```
export const deletePhoto = async (req, res, next) => {
  try {
    const photo = await FieldPhoto.findById(req.params.photoId);
    if (!photo) throw new AppError('Photo not found', 404);
    await storage.remove(photo.filePath);
    await photo.deleteOne();
    await audit.log(req.user._id, 'PHOTO_DELETED', 'FieldPhoto', photo._id);
    res.json({ success: true, message: 'Photo removed' });
  } catch (err) {
    next(err);
  }
};
```

A.1 Express Routes

```
// server/routes/index.js
app.use('/api/auth', authRoutes);
app.use('/api/users', userRoutes);
app.use('/api/contracts', contractRoutes);
app.use('/api/photos', photoRoutes);
app.use('/api/disputes', disputeRoutes);
app.use('/api/notifications', notificationRoutes);
```

A.2 Main Server File

```
import 'dotenv/config';
import express from 'express';
import cors from 'cors';
import helmet from 'helmet';
import mongoSanitize from 'express-mongo-sanitize';
import rateLimit from 'express-rate-limit';
import { connectDB } from './config/db.js';
import { errorHandler } from './middleware/error.js';
const app = express();
app.use(helmet());
app.use(cors({ origin: process.env.CLIENT_URL }));
app.use(express.json({ limit: '100kb' }));
app.use(mongoSanitize());
app.use('/api', rateLimit({ windowMs: 15 * 60 * 1000, max: 300 }));
// routes mounted here (see A.13)
app.use(errorHandler);
const port = process.env.PORT || 5000;
connectDB().then(() => app.listen(port, () => console.log('Server on ' + port)));
```

A.3 React Login Component

```
export default function Login() {
  const { t } = useTranslation();
  const { login } = useAuth();
  const [phone, setPhone] = useState('');
  const [password, setPassword] = useState('');
  const [errors, setErrors] = useState({});
  async function handleSubmit() {
    try {
      await login(phone, password);
    } catch (err) {
      const list = err.response?.data?.errors || [];
      setErrors(Object.fromEntries(list.map(e => [e.field, e.message])));
    }
  }
  return (
    <div className='page'>
      <h1>{t('login.title')}</h1>
      <input value={phone} onChange={e => setPhone(e.target.value)}
          placeholder={t('login.phone')} inputMode='numeric' />
      {errors.phone && <p className='error'>{errors.phone}</p>}
      <input type='password' value={password} onChange={e =>
          setPassword(e.target.value)} placeholder={t('login.password')} />
      <button className='btn-large'
          onClick={handleSubmit}>{t('login.submit')}</button>
    </div>
  );
}
```

A.4 Admin Dashboard Logic

```
useEffect(() => {
  api.get('/users/pending').then(r => setPending(r.data.data));
  api.get('/disputes?status=open').then(r => setDisputes(r.data.data));
}, []);
async function verify(id) {
  await api.patch('/users/' + id + '/verify');
  setPending(list => list.filter(u => u._id !== id));
}
async function resolve(id, note) {
  await api.patch('/disputes/' + id + '/resolve', { resolutionNote: note });
  setDisputes(list => list.filter(d => d._id !== id));
}
```

A.5 REST API Request and Response Examples

Example 1: login request.

```
POST /api/auth/login
{
  "phone": "9876543210",
  "password": "<password>"
}
```

Example 2: successful login response.

```
200 OK
{
  "success": true,
  "data": {
    "token": "<jwt>",
    "user": { "id": "<id>", "name": "<name>", "role": "farmer", "language": "ta",
        "verified": true }
  }
}
```

Example 3: create contract request.

```
POST /api/contracts
Authorization: Bearer <token>
{
  "farmer": "<farmer id>",
  "crop": "Paddy",
  "quantityQuintals": 90,
  "pricePerQuintal": 2400,
  "grade": "A",
  "deliveryDate": "2027-01-24"
}
```

Example 4: validation error response (returned by the server when fields are missing or wrong).

```
{
  "success": false,
  "message": "Validation failed",
  "errors": [
    { "field": "farmer", "message": "Farmer ID is required" },
    { "field": "quantityQuintals", "message": "Quantity must be greater than 0" }
  ]
}
```

Example 5: role error response.

```
{
  "success": false,
  "message": "Validation failed",
  "errors": [
    { "field": "role", "message": "Role must be one of farmer, buyer, or admin" }
  ]
}
```

Example 6: payment release response. For the example contract above the total is 90 x 2400 = 216000, so milestone 0 is 30% = 64800.

```
POST /api/contracts/<id>/payments/0/release
200 OK
{
  "success": true,
  "data": { "milestone": 0, "percent": 30, "amount": 64800, "status": "paid",
      "invoiceNo": "INV-CF-1042-<timestamp>" }
}
```

Add one successful response for each remaining endpoint from your own run (copy it from the browser's Network tab, Response view).

SCREENSHOTS

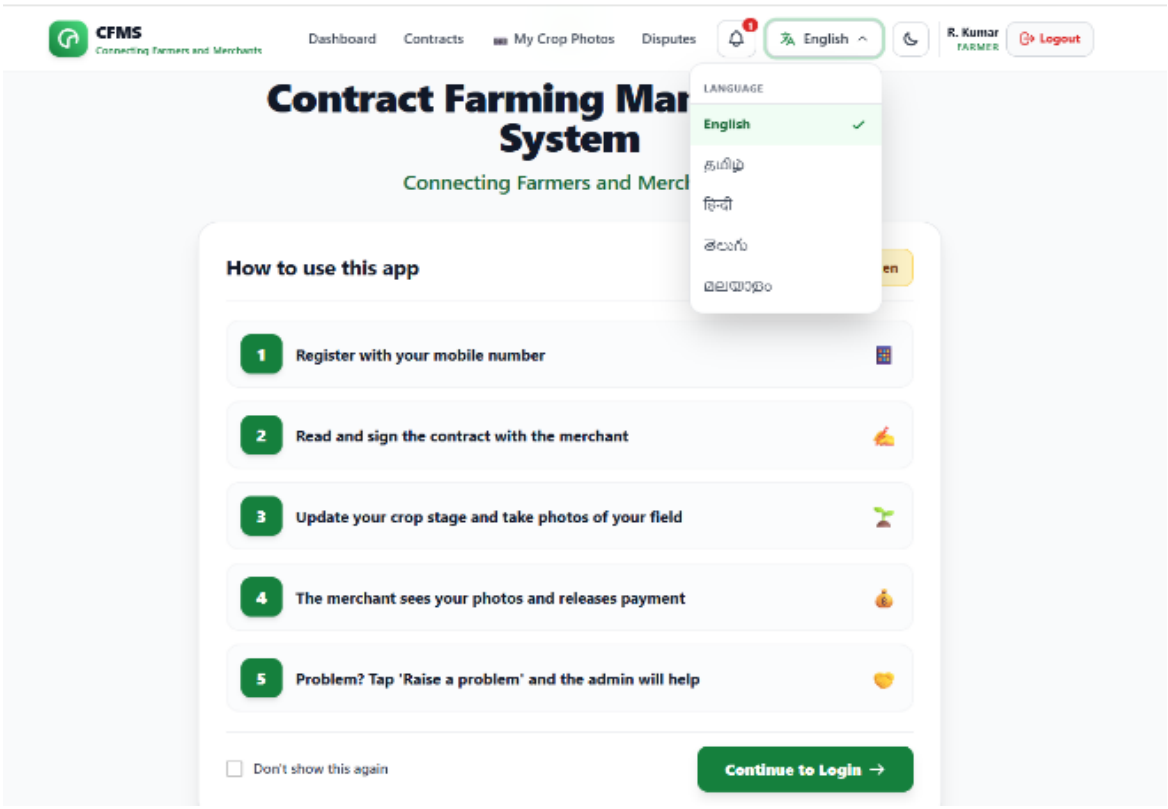


Figure 7.1: Welcome page with the five language buttons

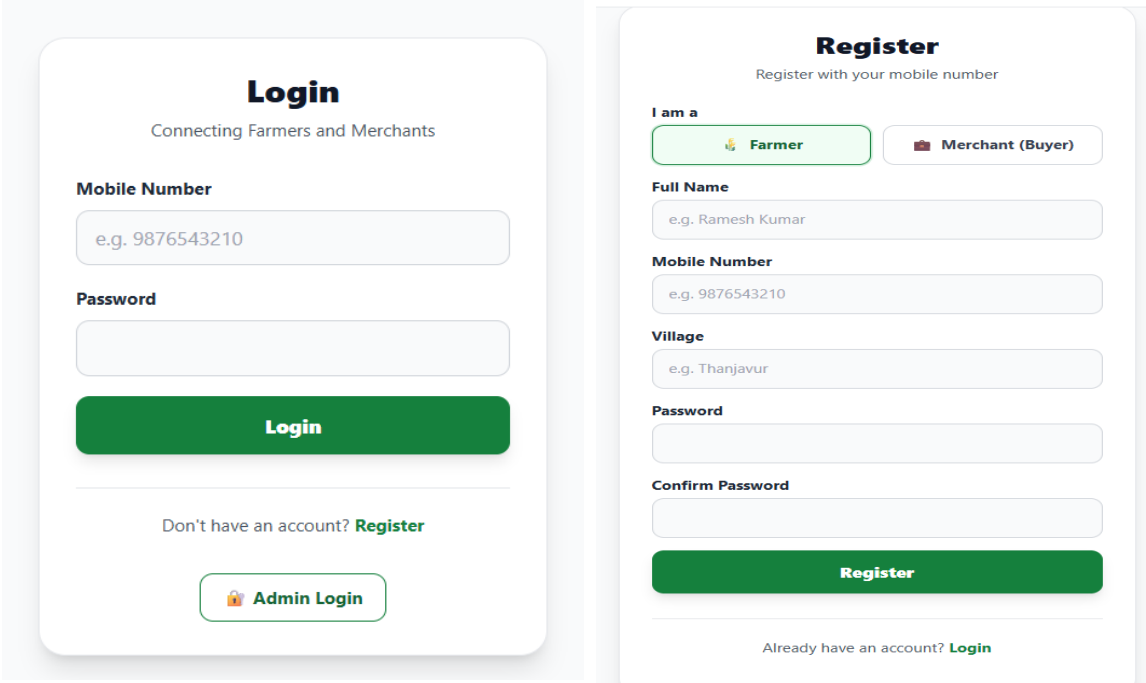


Figure 7.2: Login and registration pages

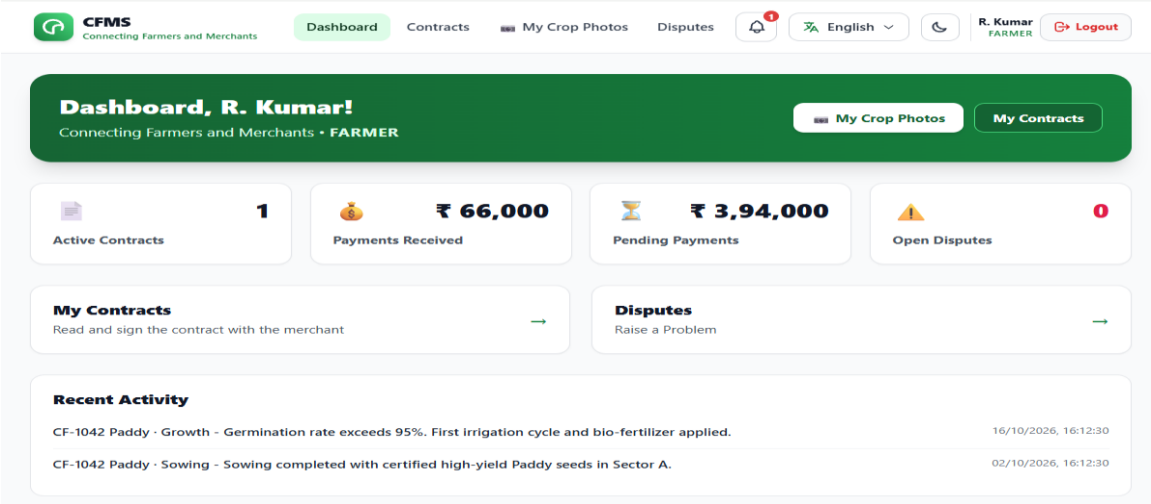


Figure 7.3: Farmer dashboard

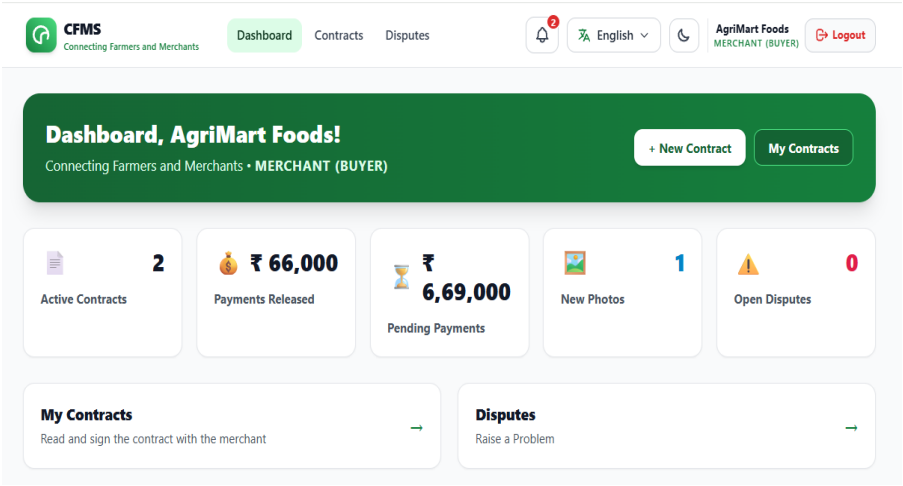


Figure 7.4: Merchant dashboard

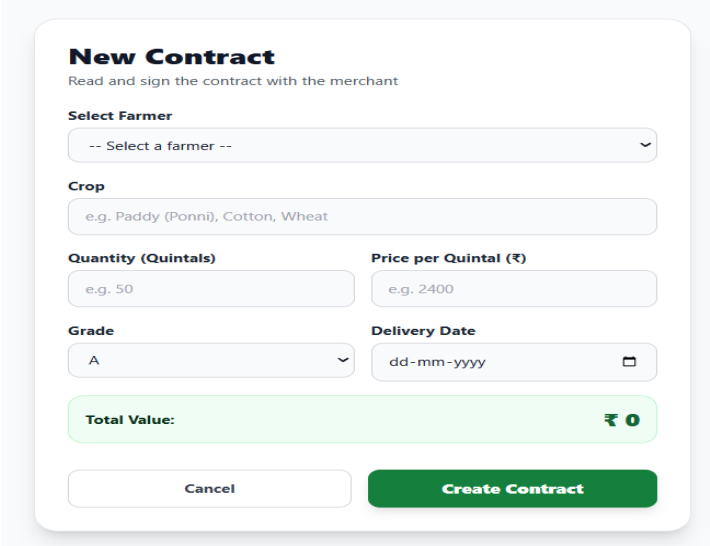


Figure 7.5: New Contract form and the Sign Contract dialog

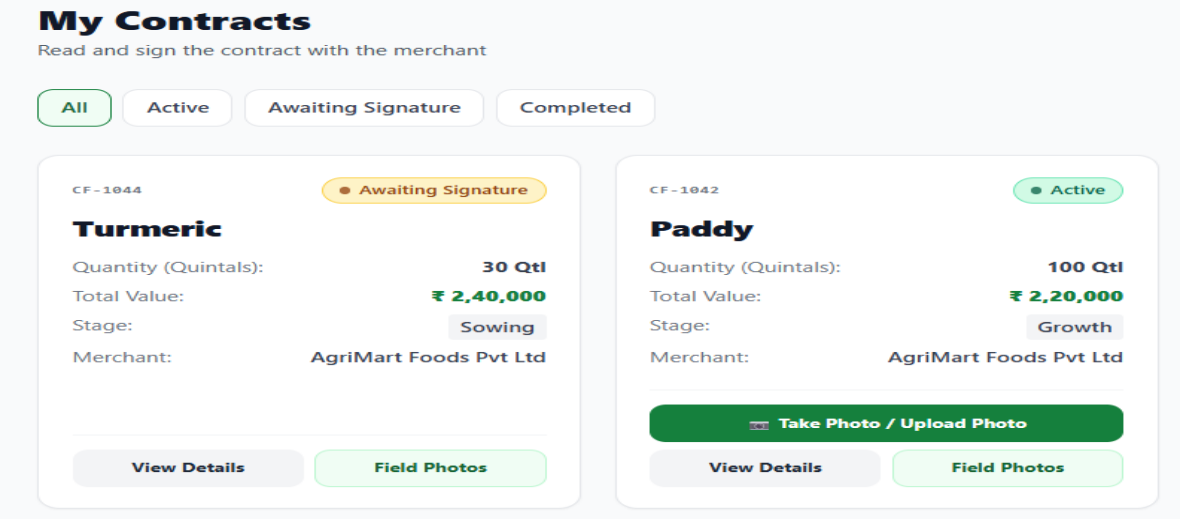


Figure 7.6: Contract details showing both signatures and active status

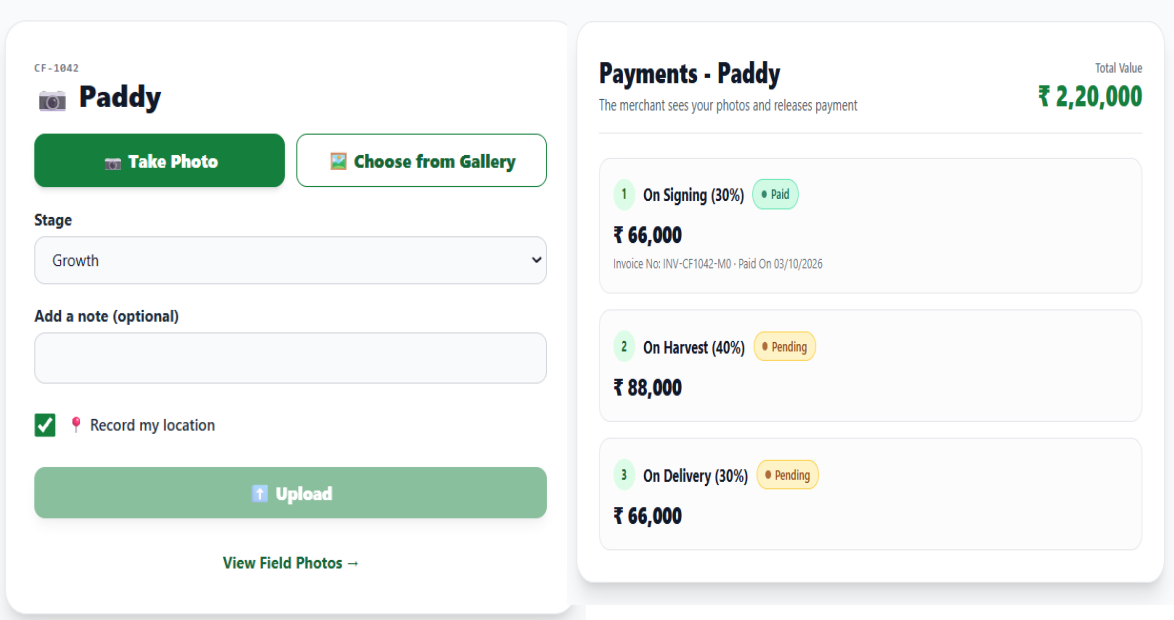


Figure 7.8: Camera capture and photo upload

Figure 7.10: Payment screen

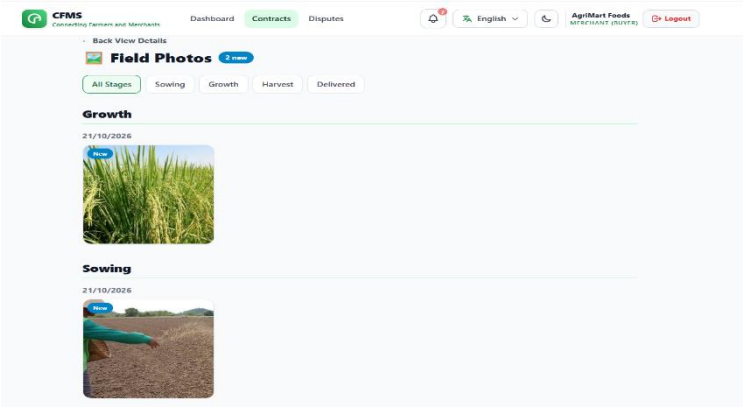


Figure 7.9: Field Photos gallery and full-screen viewer (merchant)

REFERENCES

[1] Government of India, Ministry of Agriculture and Farmers Welfare, Model Agricultural Produce and Livestock Contract Farming and Services (Promotion and Facilitation) Act, 2018, 2018.

[2] Government of India, The Farmers (Empowerment and Protection) Agreement on Price Assurance and Farm Services Act, 2020, Gazette of India, 2020 (repealed in 2021).

[3] National Agriculture Market (e-NAM), Government of India. https://enam.gov.in

[4] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015.

[5] N. Provos and D. Mazières, "A Future-Adaptable Password Scheme," in Proc. USENIX Annual Technical Conference, 1999.

[6] MongoDB Inc., MongoDB Atlas Documentation. https://www.mongodb.com/docs/atlas/

[7] Mongoose, Mongoose Documentation. https://mongoosejs.com

[8] OpenJS Foundation, Express.js Documentation. https://expressjs.com

[9] Meta Open Source, React Documentation. https://react.dev

[10] Mozilla Developer Network, "MediaDevices: getUserMedia() method." https://developer.mozilla.org/en-US/docs/Web/API/MediaDevices/getUserMedia

[11] Mozilla Developer Network, "Web Speech API." https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API

[12] OWASP Foundation, OWASP Top 10 Web Application Security Risks. https://owasp.org/www-project-top-ten/

[13] Razorpay, Razorpay Payments Documentation. https://razorpay.com/docs/

Add the peer-reviewed papers from Section 2.1 here, and the date you accessed each web page, in the format your department requires.